Fundamentos Sobre Engenharia de Software

Introdução

Prof.ª: Tamiris

Unoeste | EAD

CONCEITOS ENGENHARIA DE SOFTWARE

- A Engenharia de Software - ES é uma disciplina da área de Ciência da Computação, que estuda todos os processos envolvidos no desenvolvimento de software (SOMMERVILLE, 2019).

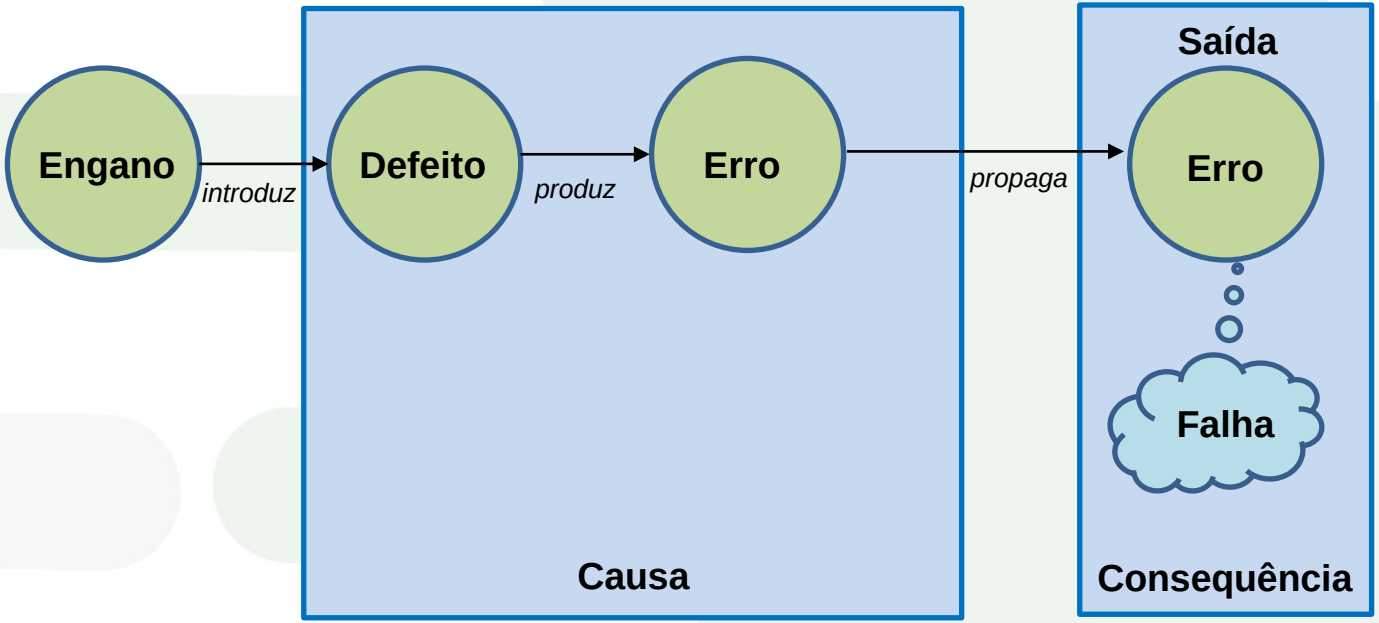
- É de fundamental importância, uma vez que, o processo de desenvolvimento de sistemas envolve diversas atividades distintas, é a ES que preocupa-se em estudar e monitorar o bom andamento de todas essas atividades (SOMMERVILLE, 2019).

- A Engenharia de Software envolve:
  - Planejamento
  - Qualidade
  - Requisitos
  - Desenvolvimento

SISTEMA DE INFORMAÇÃO BASEADO EM COMPUTADOR - SIBC

- **SIBC:** é um conjunto ou disposição de elementos que é organizado para executar certo método, procedimento ou controle ao processar informações (SOMMERVILLE, 1995).

- Elementos que compõem um SIBC:


Engano
introduz
Defeito
produz
Erro
Causa
propaga
Saída
Erro
Falha
Consequência

ONDE UTILIZAMOS SOFTWARE?
(Aplicações do Software)

- **BÁSICO:** coleção de programas escritos para dar apoio a outros programas (compiladores, componentes de sistemas operacionais).
- **DE TEMPO REAL:** software que monitora, analisa e controla eventos do mundo real.
- **COMERCIAL:** sistemas de operações comerciais e tomadas de decisões administrativas.
- **CIENTÍFICO E DE ENGENHARIA:** caracterizado por algoritmos de processamento de números (biologia molecular, manufatura automatizada).
- **EMBUTIDO:** reside situado nas memórias ROM e é usado para controlar produtos e sistemas para os mercados industriais e de consumo (teclado para forno microondas).
- **DE COMPUTADOR PESSOAL:** envolve processamento de textos, planilhas eletrônicas, diversões, etc.
- **DE INTELIGÊNCIA ARTIFICIAL:** faz uso de algoritmos não numéricos para resolver problemas que não sejam favoráveis à computação ou à análise direta (sistemas especialistas, de reconhecimento de padrões de imagem e voz, de redes neurais artificiais, de jogos, etc).
- **PARA WEB:** refere-se ao software que incorpora as instruções executáveis (CGI, HTML, Perl, Java) e dados (hipertexto, formatos visuais e de áudio).

EVOLUÇÃO E CRISE DO SOFTWARE

Crise de Software

Refere-se a um conjunto de problemas encontrados no desenvolvimento de software.

PROBLEMAS?

Crise de Software

Refere-se a um conjunto de problemas encontrados no desenvolvimento de software.

- As estimativas de prazos e de custo, frequentemente imprecisas.
- Insatisfação do cliente com o resultado final.
- A qualidade de software, às vezes, é menos que adequada.
- O software existente é muito difícil de manter.

PORQUE?

Crise de Software

Refere-se a um conjunto de problemas encontrados no desenvolvimento de software.

O PRÓPRIO CARÁTER DO SOFTWARE.

- O software é um elemento de sistema lógico e não físico.
- O software não se desgasta, mas deteriora.

FALHAS DAS PESSOAS RESPONSÁVEIS PELO DESENVOLVIMENTO DE SOFTWARE

- Falta de experiência.
- Não existia treinamento formal, profissional, muita resistência a mudanças.

ENGENHARIA DE SOFTWARE

A aplicação de uma **abordagem sistemática**, **disciplinada** e **possível de ser medida** para o desenvolvimento, operação e manutenção do software.

O QUE É UM PROCESSO DE SOFTWARE?

A aplicação de uma **abordagem sistemática**, disciplinada e possível de ser medida para o desenvolvimento, operação e manutenção do software.

Processo:

“Qualquer atividade, ou conjunto de atividades, que usa recursos para transformar insumos (entrada) em produtos (saídas).”

(NBR ISSO 9000, 2000)

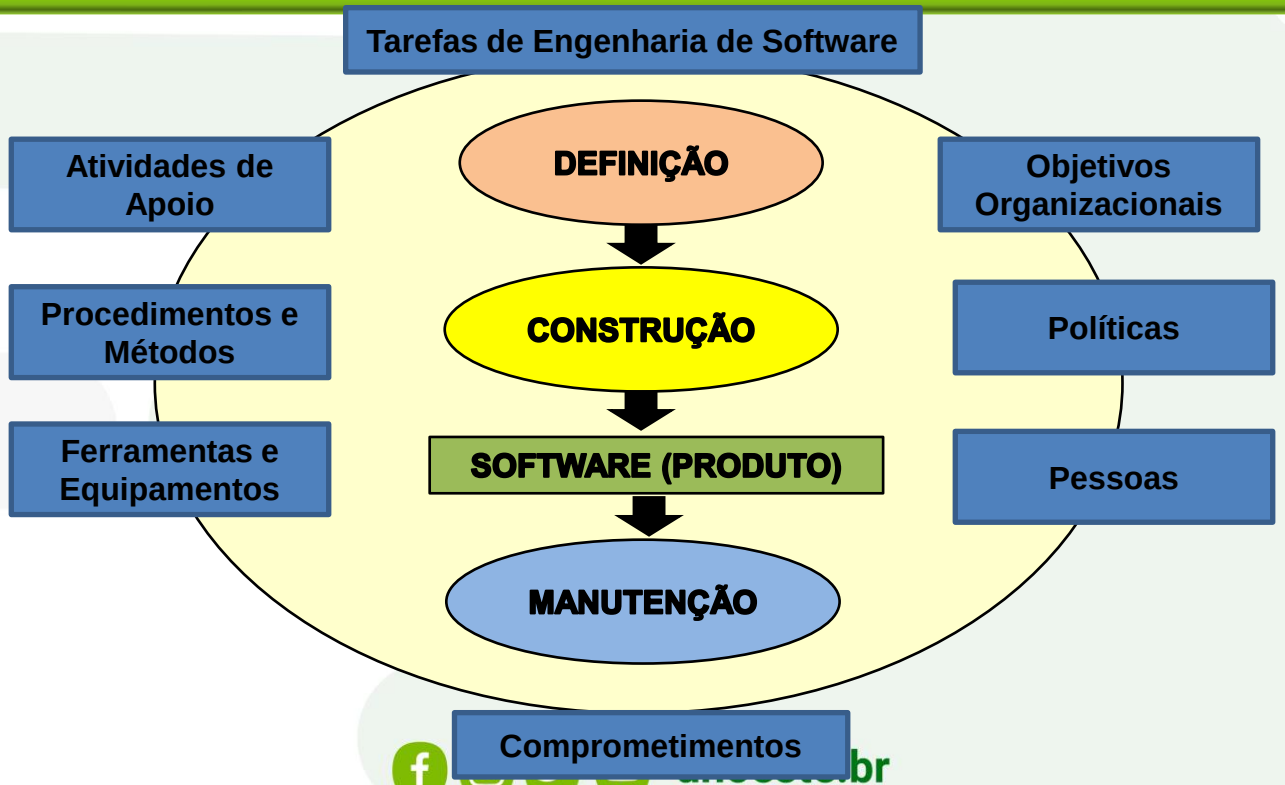
ALGUMAS CARACTERÍSTICAS DE PROCESSO

- Segundo Pfleeger (2004), um processo:
  - Prescreve todas as suas principais atividades.
  - Utiliza recursos.
  - Está sujeito a um conjunto de restrições (ex: um cronograma).
  - Gera produtos intermediários e finais.
  - Pode ser composto de subprocesso relacionados de algum modo.

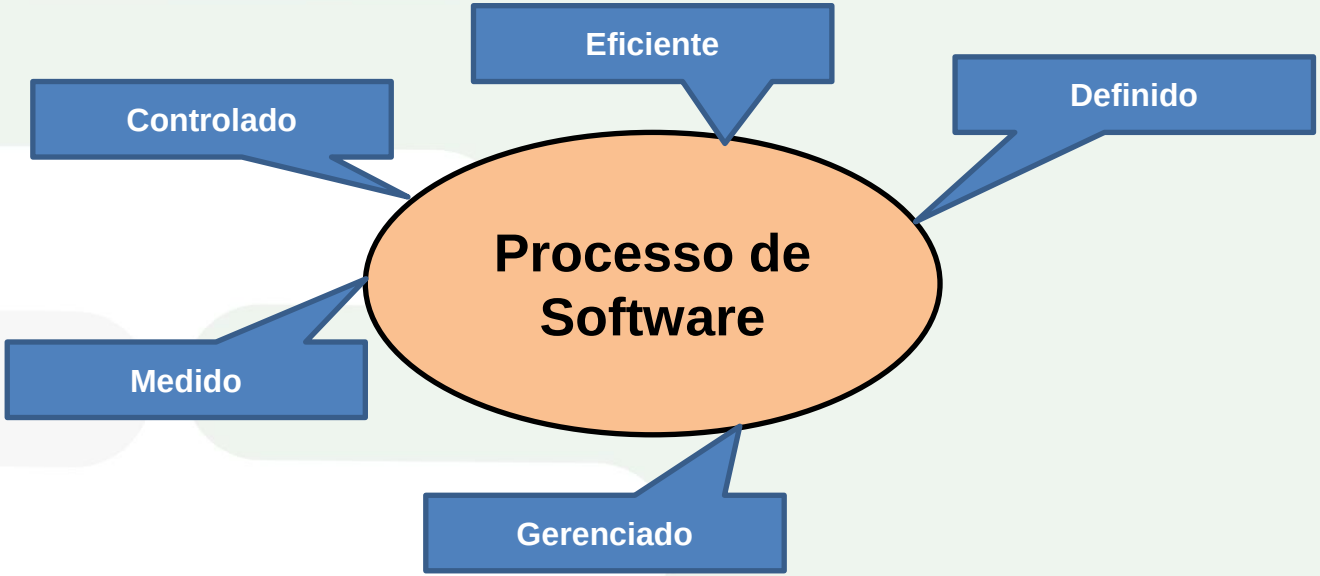
PROCESSO DE SOFTWARE

- Para que funcione satisfatoriamente, deve possuir:
  - Procedimentos e Métodos
    - Descrevem a relação entre as tarefas.
  - Ferramentas e equipamentos.
    - Dão suporte à realização das tarefas, simplificando e automatizando o trabalho.
  - Pessoas.
    - Realizam as tarefas apropriadamente, para tanto, possuem perfil adequado e são treinadas nos métodos e ferramentas adotadas (Paulk, 1995).

ELEMENTOS ENVOLVIDOS NO PROCESSO DE SOFTWARE

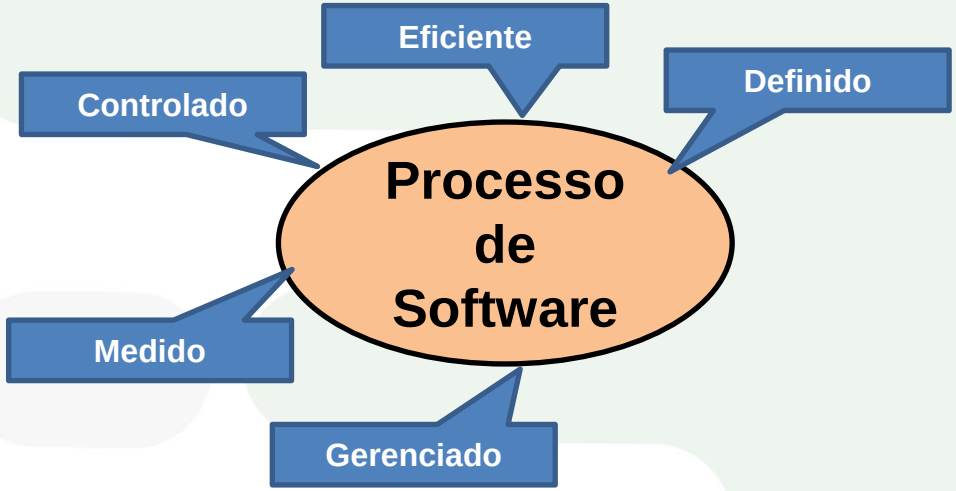


PROCESSO E QUALIDADE



O que é utilizado para que um processo possa ser definido, eficiente, controlado, medido e gerenciado?

PROCESSO DE SOFTWARE



Modelo de Processo de Software

MODELOS DE PROCESSO DE SOFTWARE

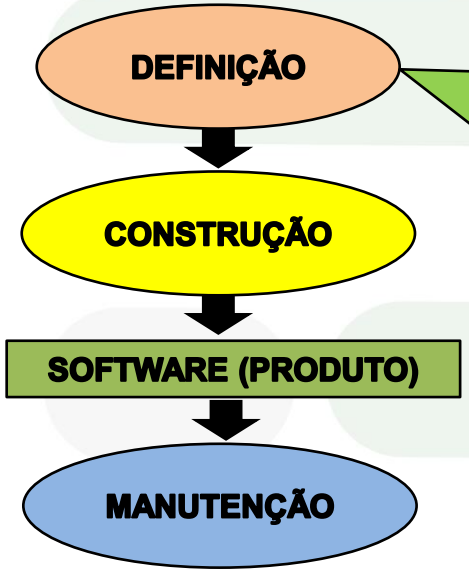
- Procuram descrever **formalmente** e de **maneira organizada** todas as atividades que devem ser seguidas para a obtenção segura de um produto de software.

- A escolha do modelo de processo de software depende:
  - da **natureza** do projeto e da aplicação.
  - dos **métodos** e **ferramentas** a serem usados.
  - dos **controles** e **produtos** que precisam ser entregues.

- Existem vários ***modelos de processo de software*** (ou *paradigmas de engenharia de software ou modelos de ciclo de vida*).

- Cada um representa uma tentativa de colocar ordem em uma atividade inerentemente caótica.

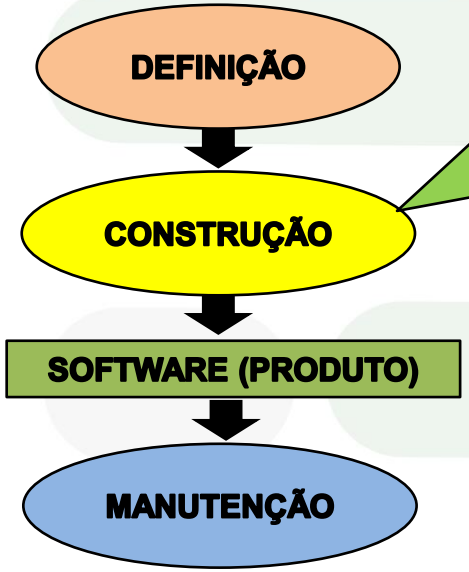
FASES GENÉRICAS



"O Que"

- **Informações** a serem processadas.
- **Funções** e **desempenho** desejados.
- **Interfaces** que devem ser estabelecidas.
- **Restrições** do projeto.
- Critérios de **validação** requeridos.

FASES GENÉRICAS



"Como"

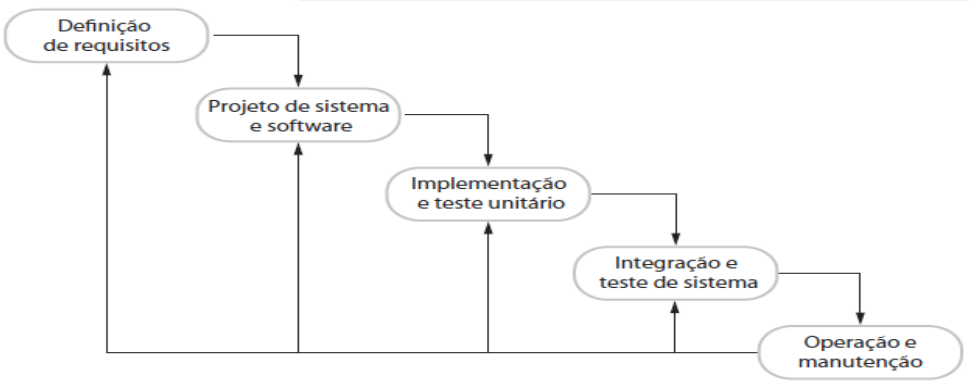
- Como devem ser projetadas a **estrutura de dados** e a **arquitetura** do software.
- Como os **procedimentos** devem ser implementados.
- Como o projeto deve ser traduzido para uma **linguagem de programação**.
- Como o **teste** deve ser executado.

MODELOS DE PROCESSO DE SOFTWARE TRADICIONAIS

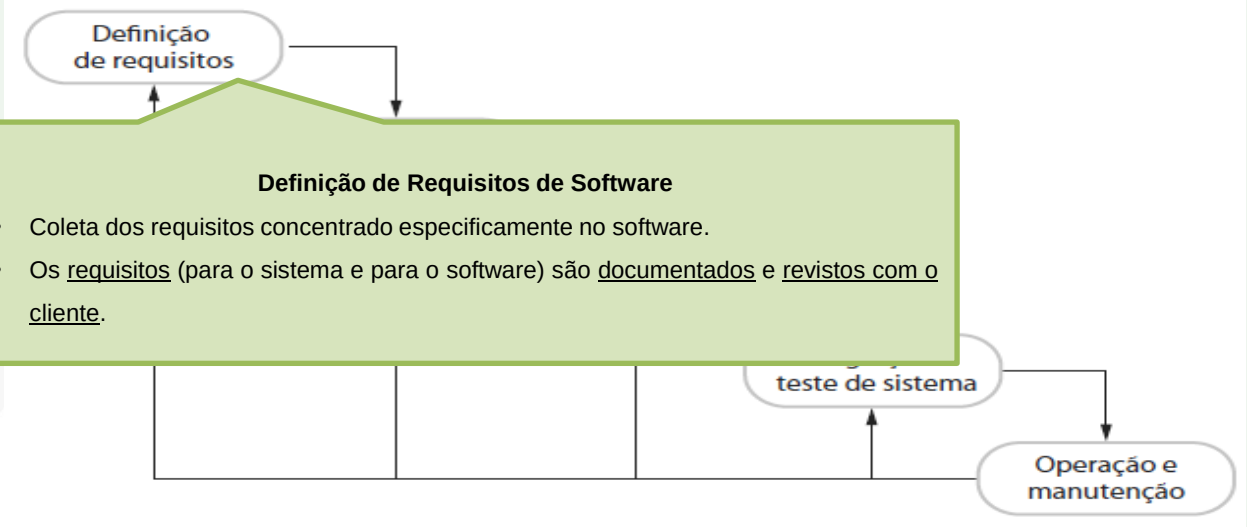
- **Alguns Modelos de Processos Tradicionais:**
  - Modelo Cascata (Modelo Sequencial Linear)
  - Modelo Prototipação
  - Modelo Incremental

CASCATA

- Modelo mais antigo e o mais amplamente usado da engenharia de software.
- Requer uma abordagem sistemática, sequencial ao desenvolvimento de software.
- O enfoque desse modelo está nos documentos e artefatos.



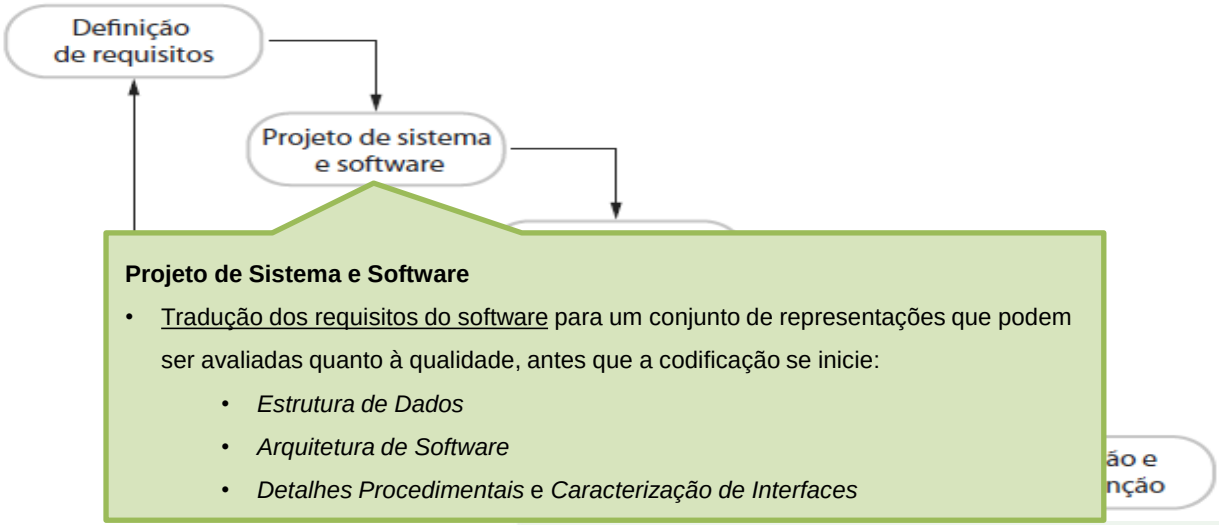
CASCATA



Definição de Requisitos de Software

- Coleta dos requisitos concentrado especificamente no software.
- Os requisitos (para o sistema e para o software) são documentados e revistos com o cliente.

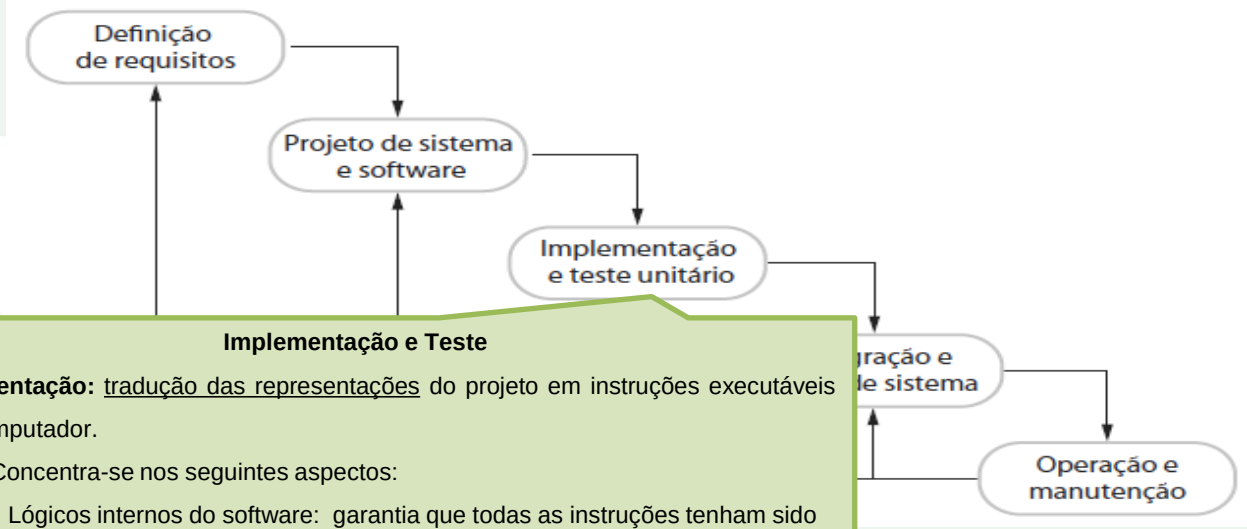
CASCATA



Projeto de Sistema e Software

- Tradução dos requisitos do software para um conjunto de representações que podem ser avaliadas quanto à qualidade, antes que a codificação se inicie:
  - *Estrutura de Dados*
  - *Arquitetura de Software*
  - *Detalhes Procedimentais* e *Caracterização de Interfaces*

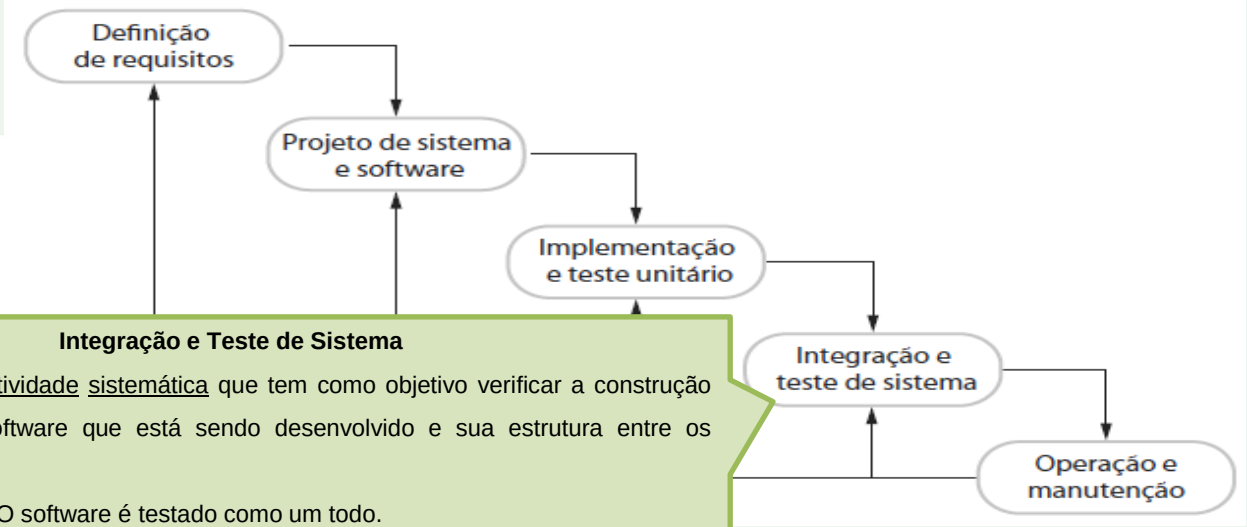
CASCATA



Implementação e Teste

- **Implementação:** <u>tradução das representações</u> do projeto em instruções executáveis pelo computador.
- **Teste:** Concentra-se nos seguintes aspectos:
  - Lógicos internos do software: garantia que todas as instruções tenham sido testadas.
  - Funcionais externos do software: garantia que a entrada definida produza resultados que concordem com os esperados.

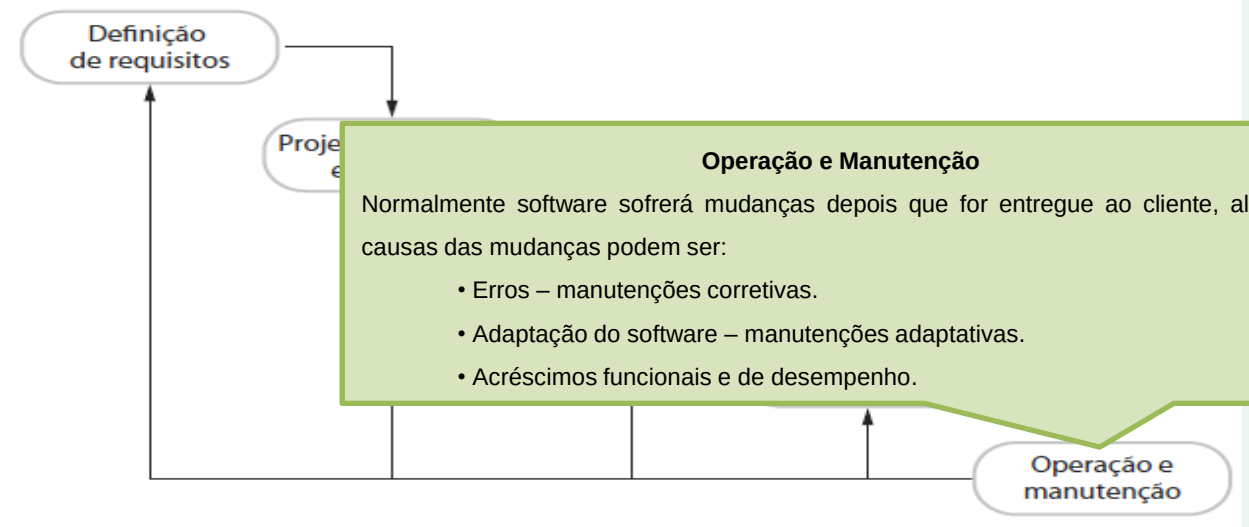
CASCATA



Integração e Teste de Sistema

- **Integração:** uma atividade sistemática que tem como objetivo verificar a construção da estrutura do software que está sendo desenvolvido e sua estrutura entre os módulos.
- **Teste de sistema:** O software é testado como um todo.
  - Testa uma release de sistema que será entregue ao cliente.
  - Verifica se os componentes são compatíveis e se interagem corretamente com a execução das funcionalidades do software.

CASCATA



Operação e Manutenção

Normalmente software sofrerá mudanças depois que for entregue ao cliente, algumas causas das mudanças podem ser:

- Erros – manutenções corretivas.
- Adaptação do software – manutenções adaptativas.
- Acréscimos funcionais e de desempenho.

PROBLEMAS DO MODELO CASCATA

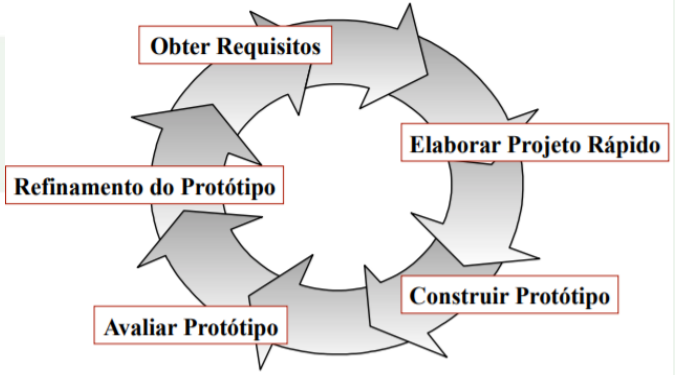
- Projetos reais raramente seguem o fluxo sequencial que o modelo propõe.

- Logo no início é difícil estabelecer explicitamente todos os requisitos → no começo dos projetos sempre existe uma incerteza natural.

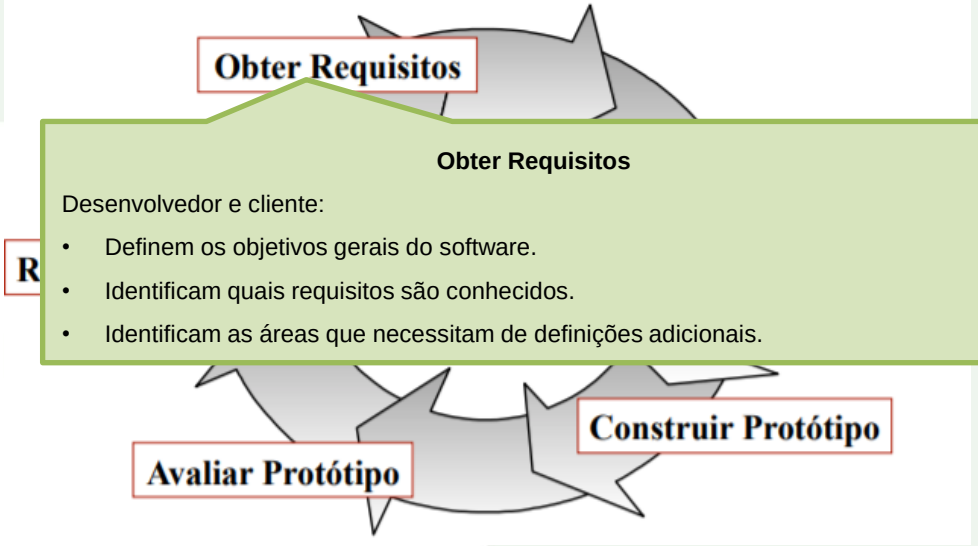
- O cliente deve ter paciência → uma versão executável do software só fica disponível numa etapa avançada do desenvolvimento.

PROTOTIPAÇÃO

- **Objetivo:** entender os requisitos do usuário.
- Possibilita que o desenvolvedor crie um modelo (protótipo) do software que deve ser construído.
- Modelo apropriado quando o cliente definiu um conjunto de objetivos gerais para o software, mas não identificou detalhadamente esses requisitos.



PROTOTIPAÇÃO

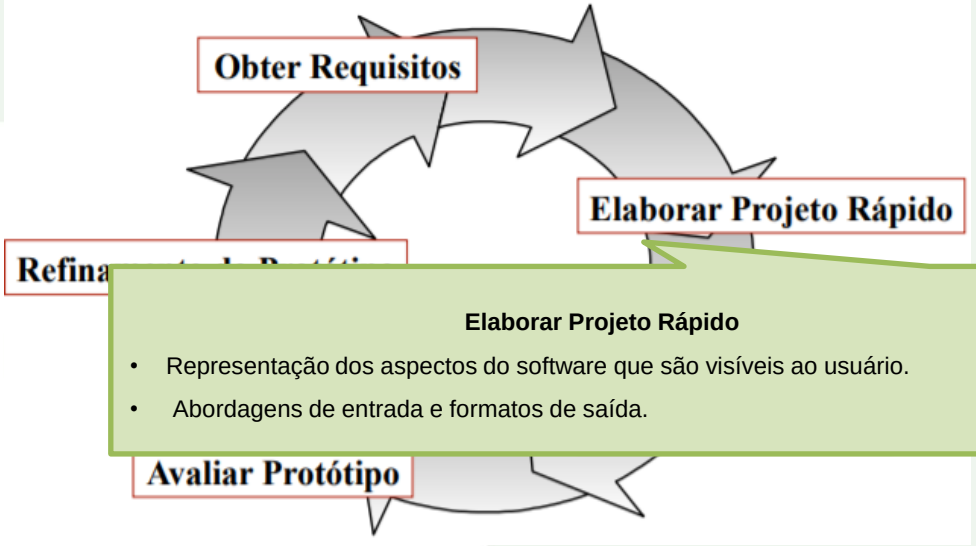


Obter Requisitos

Desenvolvedor e cliente:

- Definem os objetivos gerais do software.
- Identificam quais requisitos são conhecidos.
- Identificam as áreas que necessitam de definições adicionais.

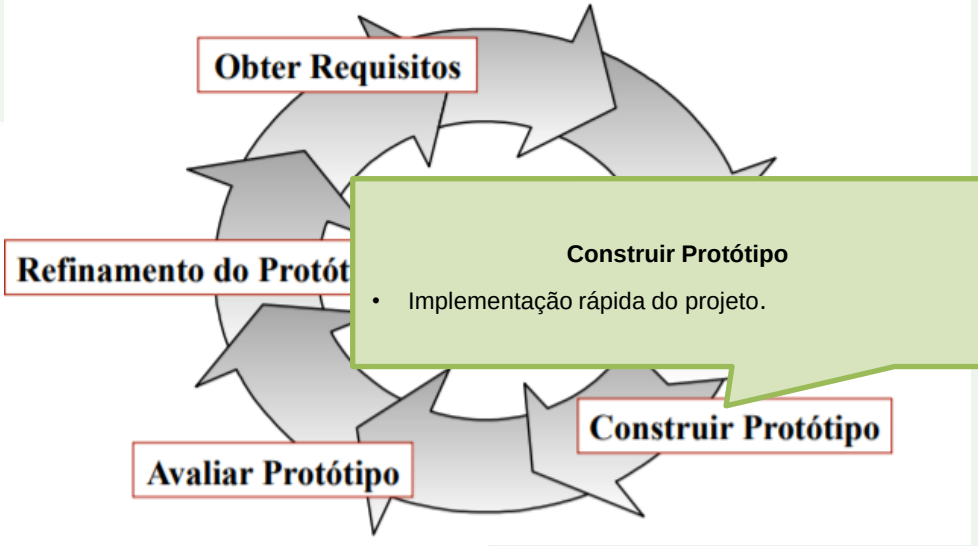
PROTOTIPAÇÃO



Elaborar Projeto Rápido

- Representação dos aspectos do software que são visíveis ao usuário.
- Abordagens de entrada e formatos de saída.

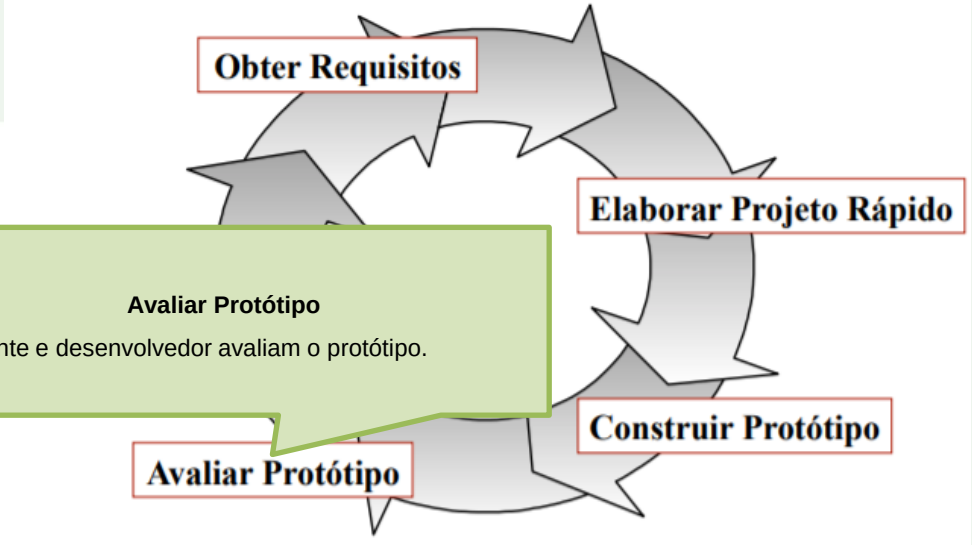
PROTOTIPAÇÃO



Construir Protótipo

- Implementação rápida do projeto.

PROTOTIPAÇÃO



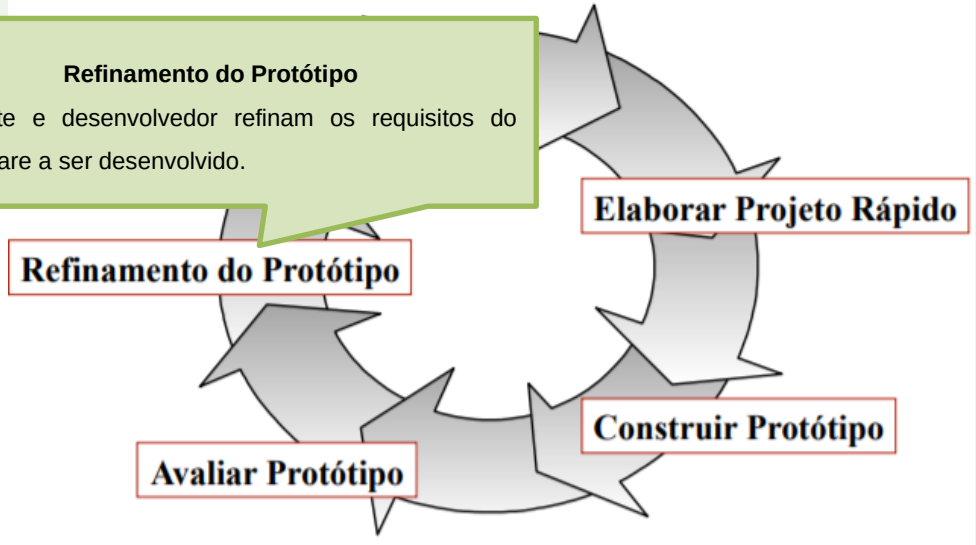
Avaliar Protótipo

- Cliente e desenvolvedor avaliam o protótipo.

PROTOTIPAÇÃO

Refinamento do Protótipo

- Cliente e desenvolvedor refinam os requisitos do software a ser desenvolvido.



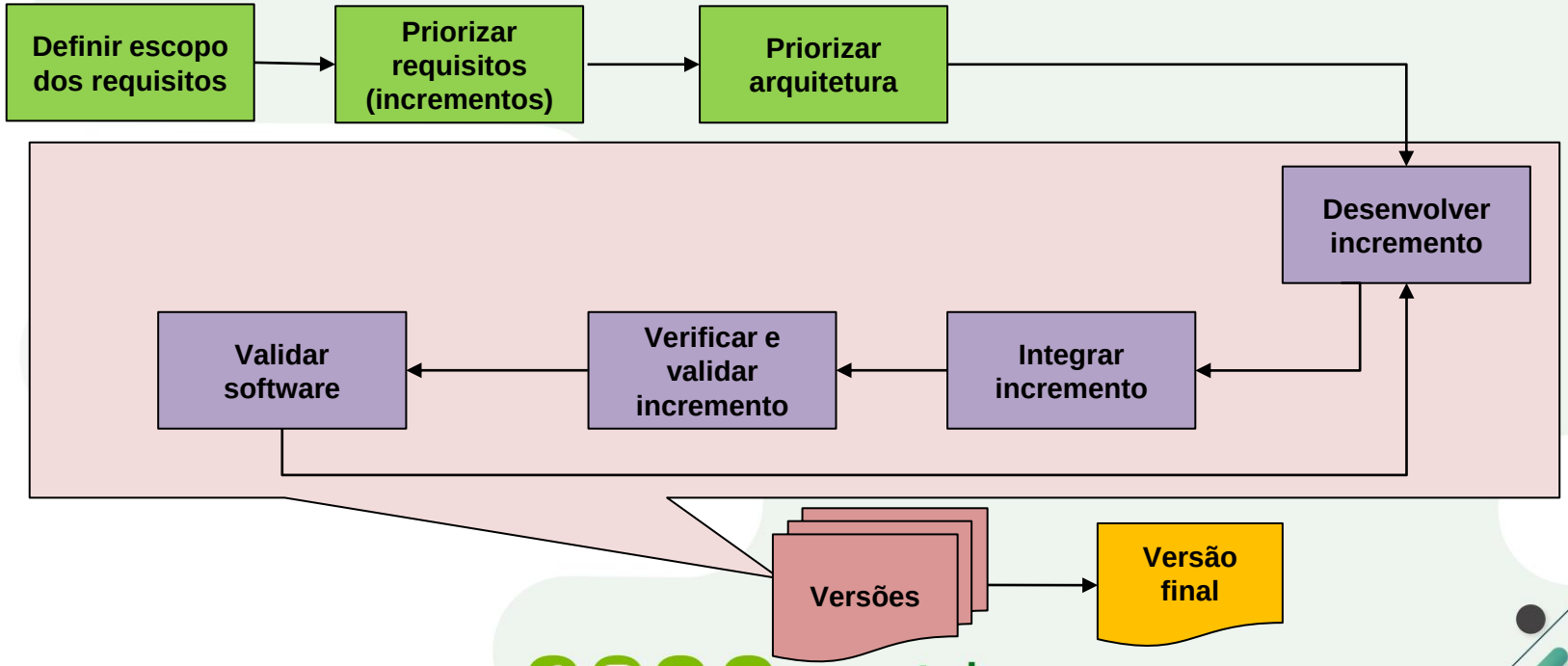
PROBLEMAS DO MODELO PROTOTIPAÇÃO

- A qualidade global e a manutenibilidade ao longo do desenvolvimento não foram consideradas.
- Utiliza-se o que há disponível na implementação com o objetivo de produzir rapidamente um protótipo → **implementação comprometida.**
- Ainda que possam ocorrer problemas, a prototipação é um ciclo de vida eficiente.
- Solução → definir as regras do jogo logo no começo.
  - Acordo entre cliente e desenvolvedor → o protótipo é construído para servir como um mecanismo a fim de definir os requisitos.

MODELO INCREMENTAL

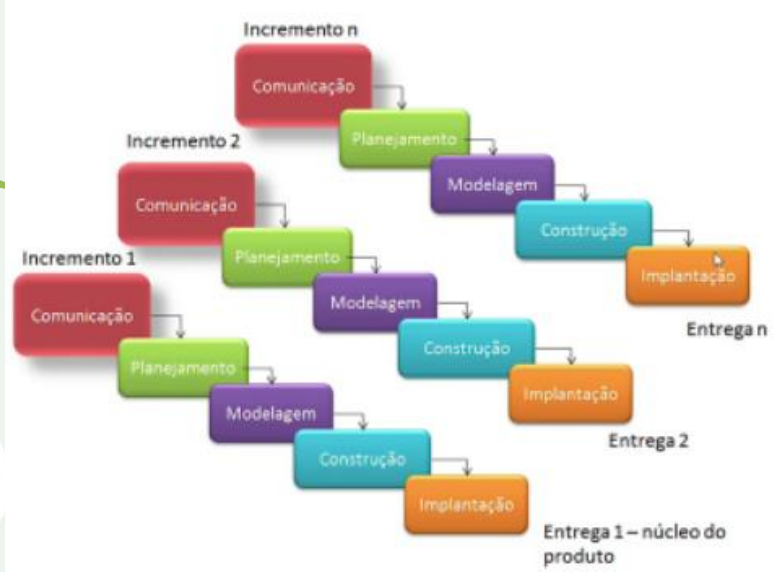
- **Objetivo:** trabalhar junto do usuário para descobrir seus requisitos, de maneira incremental, até que o produto final seja obtido.
- Tem os benefícios do modelo cascata e da prototipação.
- Ideia básica: desenvolvimento (projeto, codificação e teste) incremental do software.
- Priorização das funcionalidades principais do software.

MODELO INCREMENTAL



INCREMENTAL

Um plano é desenvolvido para o próximo incremento, como resultado do uso e/ou avaliação por parte do cliente.



VANTAGENS DO MODELO INCREMENTAL

- Os clientes não precisam esperar que a versão final do software seja entregue apara usá-lo.
- Os clientes podem utilizar os primeiros incrementos desenvolvidos como um protótipo, de forma a definir melhor alguns requisitos do software.
- Existe um risco menor de fracasso do software.
- Como as funções prioritárias são entregues primeiro, é inevitável que estas passem por um período de testes mais intensivo.

POR QUE METODOLOGIA ÁGIL?

- Problemas com o Modelo Clássico
  - *Feedback* tardio
    - Resultado final demora e muitas das vezes necessita de ajustes para se adequar a necessidade do cliente.
  - Ausência de colaboração
    - Cada membro da equipe preocupa-se apenas com seu trabalho
    - Conhecimento não compartilhado
  - Inflexibilidade
    - Falta de capacidade para responder às mudanças de forma ágil
      - Retrabalho
    - Não aceita mudanças tenta prever o futuro
  - Orientação a documentos
    - Transição no ciclo de vida por aprovação de documentos
    - Documento é utilizado para servir de defesa contra o cliente e não para produzir valor para o cliente

MÉTODOS ÁGEIS

- **Modelos de Processos Ágeis**
  - Agilidade é a habilidade tanto para criar quanto para responder à mudança de forma a obter lucro em um ambiente turbulento de negócios.
    - Visam diminuir a quantidade de documentação;
    - Tem enfoque nas pessoas e suas interações.

Atenção: Embora o nome traga a palavra ágil, a ideia está centrada na possibilidade de dividir o projeto em partes menores com entregas parciais e incrementais. Neste sentido, ser ágil é entregar valor mais rápido ao cliente.

MÉTODOS ÁGEIS

- **Alguns exemplos de Modelos de Processos Ágeis:**
  - SCRUM
  - XP (*Extreme Programming*)
  - AUP, Família CRYSTAL
  - FDD (*Feature-Driven Development*)
  - DSDM (*Dynamic Systems Development Method*)
  - ASD (*Adaptive Software Development*)
  - Kanban

SCRUM

- Ambiente em constante mudança.
- Aceitação de mudanças.
- Focado em planejamento contínuo.
- Entregas constantes.
- Equipes pequenas.
- Dividido em três fases principais que são subdivididas.

SCRUM

Pré-Planejamento

- Requisitos descritos no Product Backlog.
- Requisitos ordenados por prioridade e estimativa de esforço para desenvolver.
- Definição de equipes, ferramentas e riscos para o desenvolvimento.

Desenvolvimento

- Riscos controlados durante o desenvolvimento.
- Desenvolvidos em sprints (iterações).
- Sprints devem durar de uma semana a um mês.
- Sprints desenvolvidos de forma tradicional (análise, projeto, implementação e teste).

Pós-Desenvolvimento

- Análise do progresso do projeto.
- Demonstrar as funcionalidades atual do sprint para o cliente.

Unoeste | EAD

SCRUM (Papéis)

PRODUCT OWNER

- Dono do produto, cliente ou seu representante.
- Responsável por definir as características do produto a ser desenvolvido.
- Identifica requisitos, tira dúvidas.
- Define a ordem de desenvolvimento.
- Disponível a todo momento, bom comunicador, entendimento do negócio, bom líder.

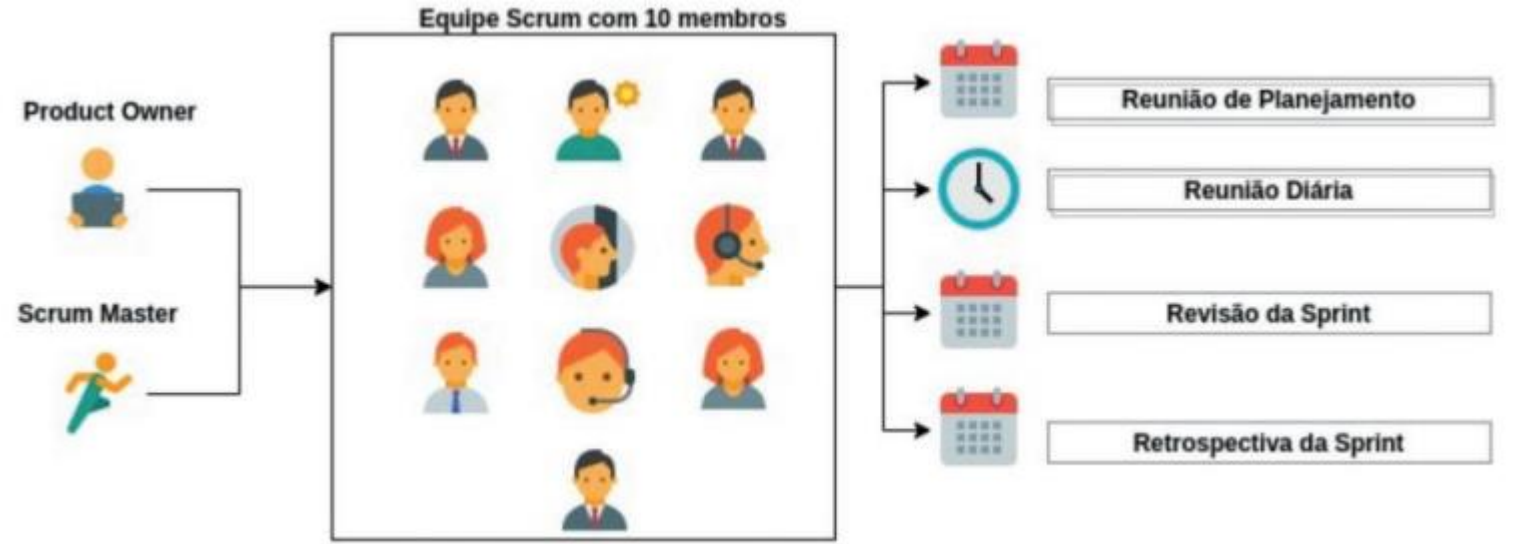
SCRUM MASTER

- Responsável pelo andamento do projeto e aplicações das práticas do Scrum.
- Busca o melhor resultado da equipe (motivador) e auxilia nos riscos.
- Mantém a equipe focada no projeto e objetivos.

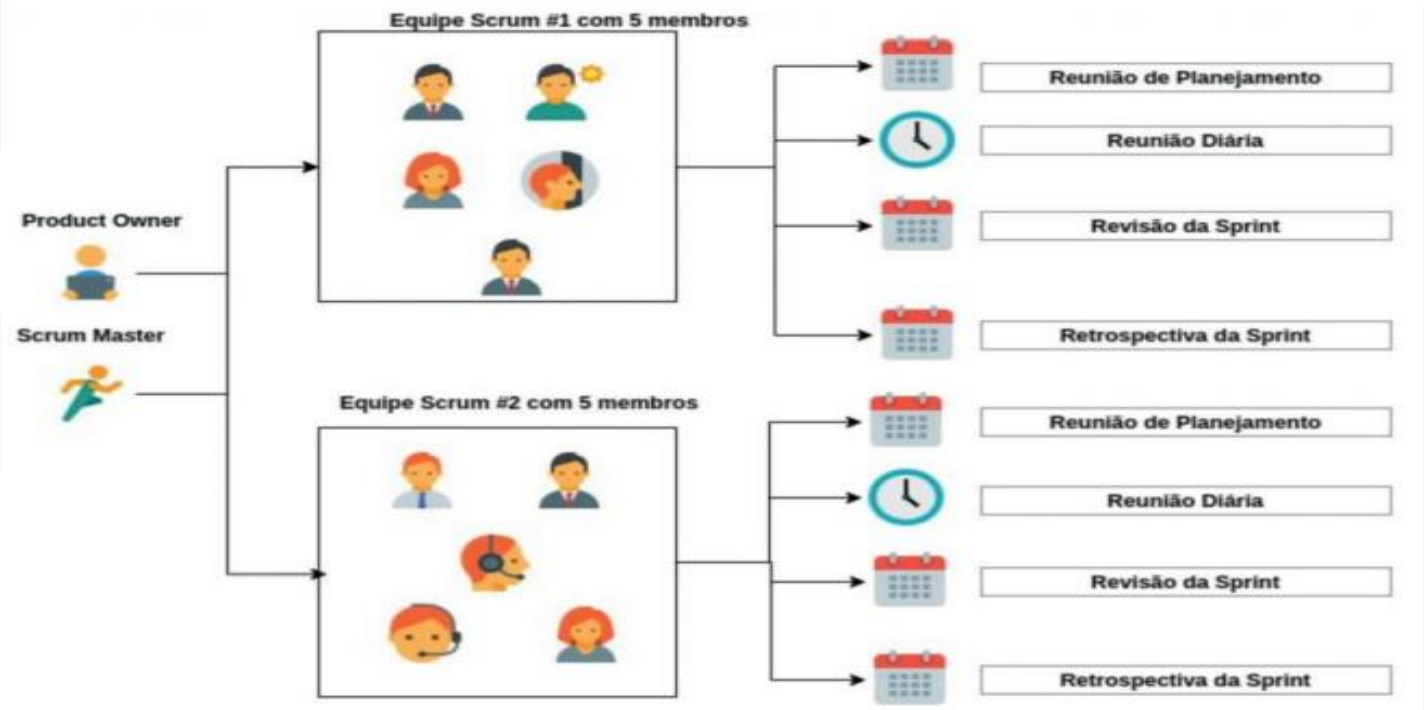
EQUIPE

- De 3 a 10 membros.
- Multi-habilidades.
  - Desenvolvedores
  - Analistas
  - Projetistas
  - Designers
  - Testadores
  - Entre outros.

SCRUM


Equipe Scrum com 10 membros
Product Owner
Scrum Master
Reunião de Planejamento
Reunião Diária
Revisão da Sprint
Retrospectiva da Sprint

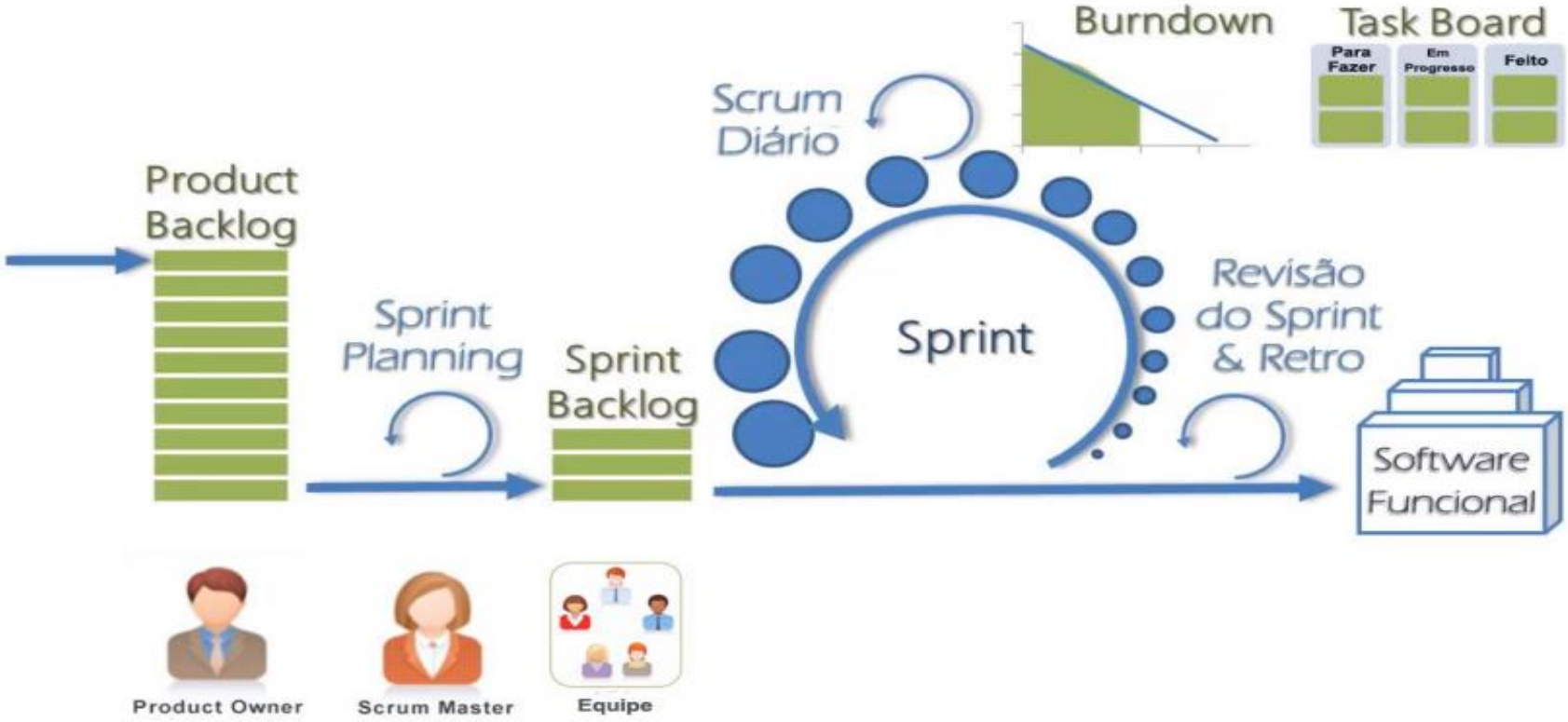
SCRUM



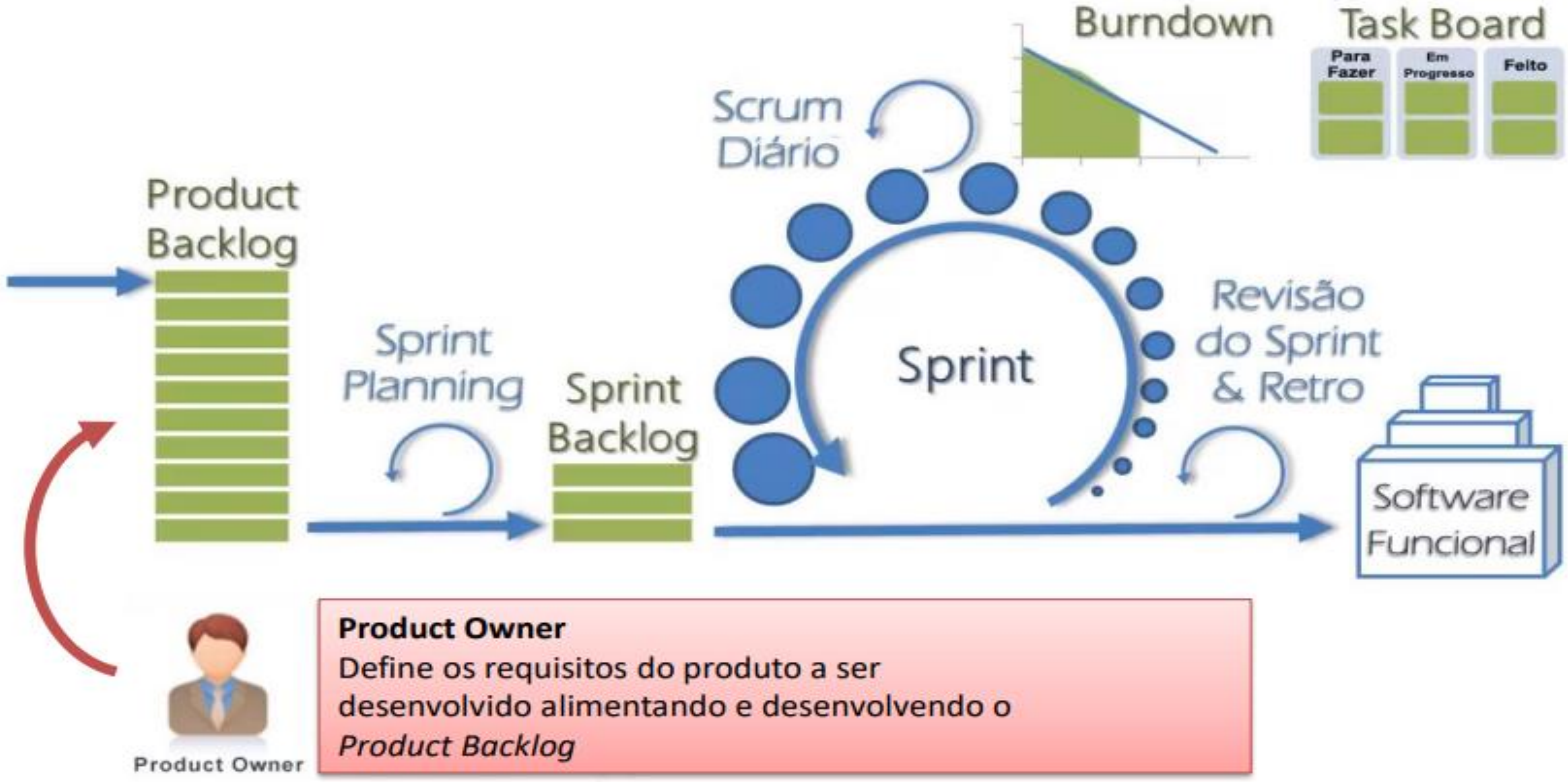
Representação de dois times Scrum com cinco membros cada.

Unoeste | EAD

CICLO DE VIDA SCRUM



CICLO DE VIDA SCRUM

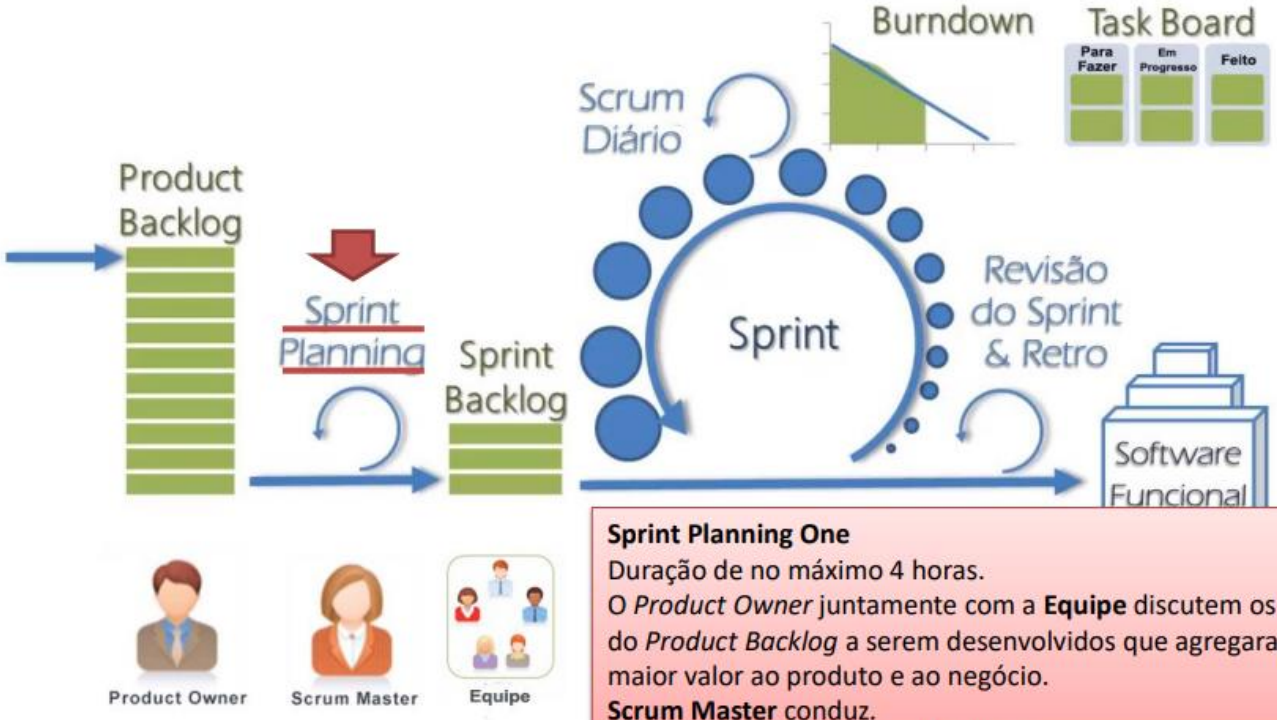


CICLO DE VIDA SCRUM

Unoeste | EAD

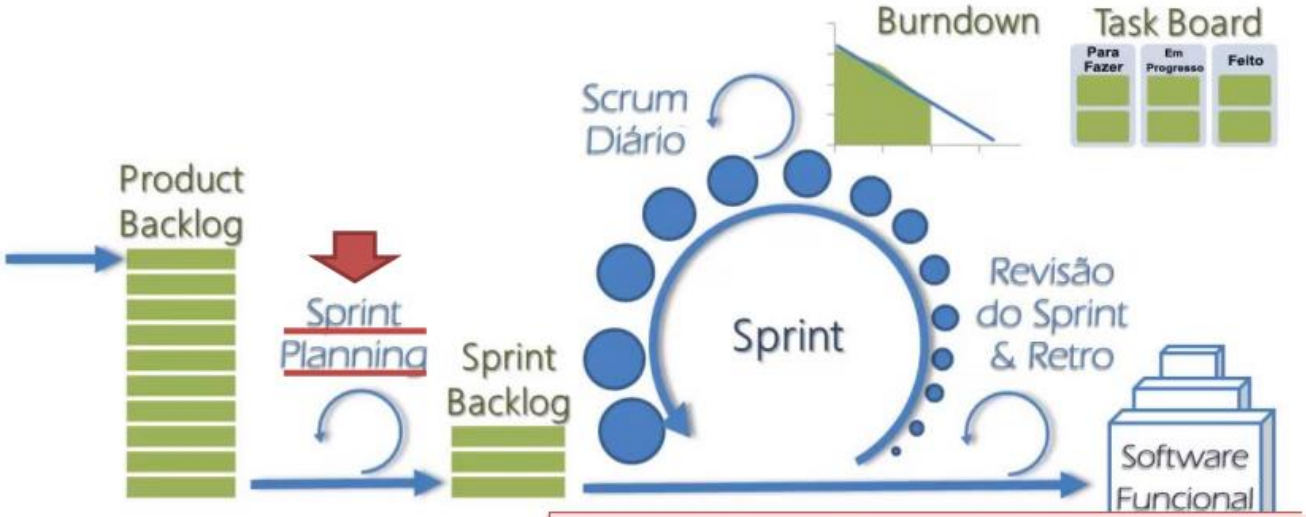
CICLO DE VIDA SCRUM

CICLO DE VIDA SCRUM



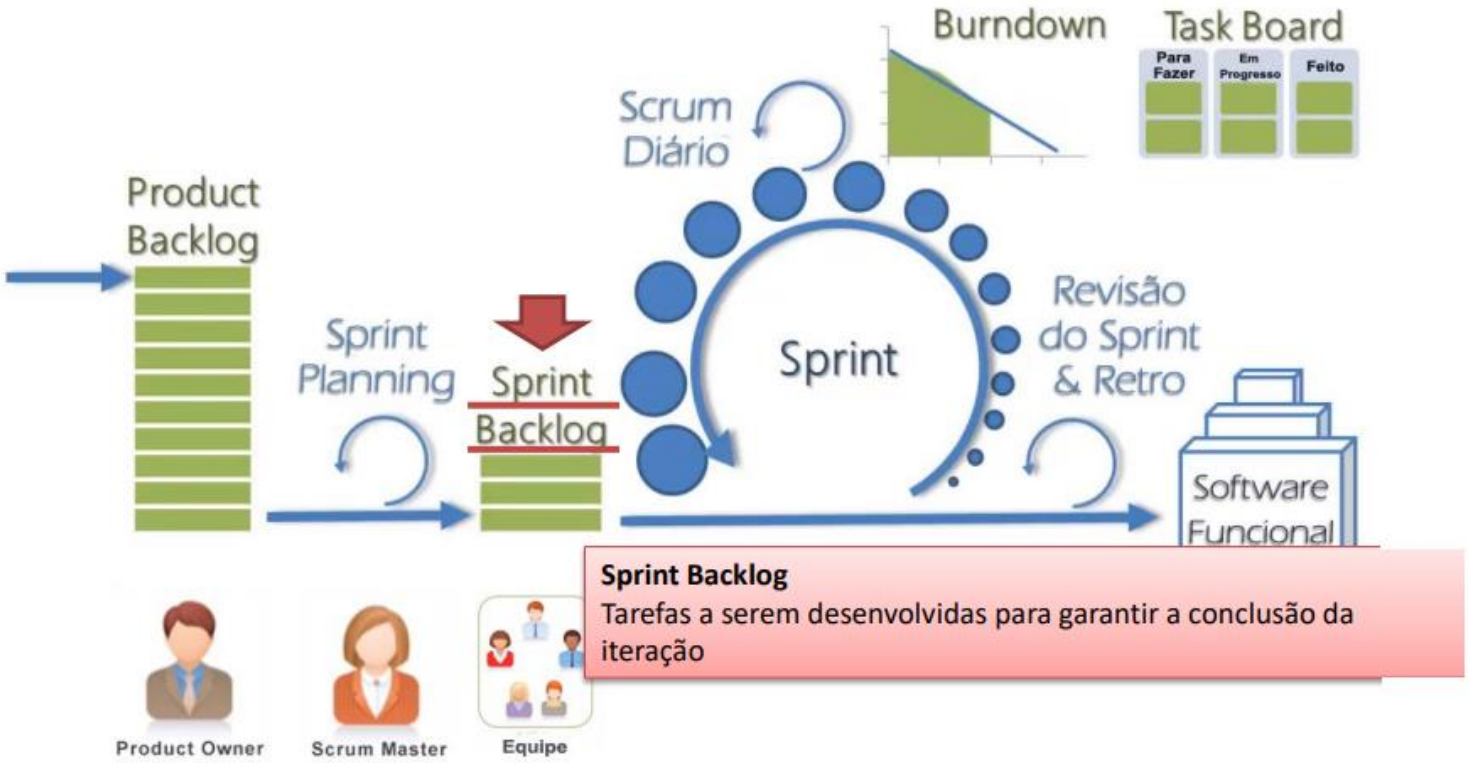
Sprint Planning One
Duração de no máximo 4 horas.
O *Product Owner* juntamente com a **Equipe** discutem os itens do *Product Backlog* a serem desenvolvidos que agregaram maior valor ao produto e ao negócio.
Scrum Master conduz.
Obtém o Selected Product Backlog

CICLO DE VIDA SCRUM



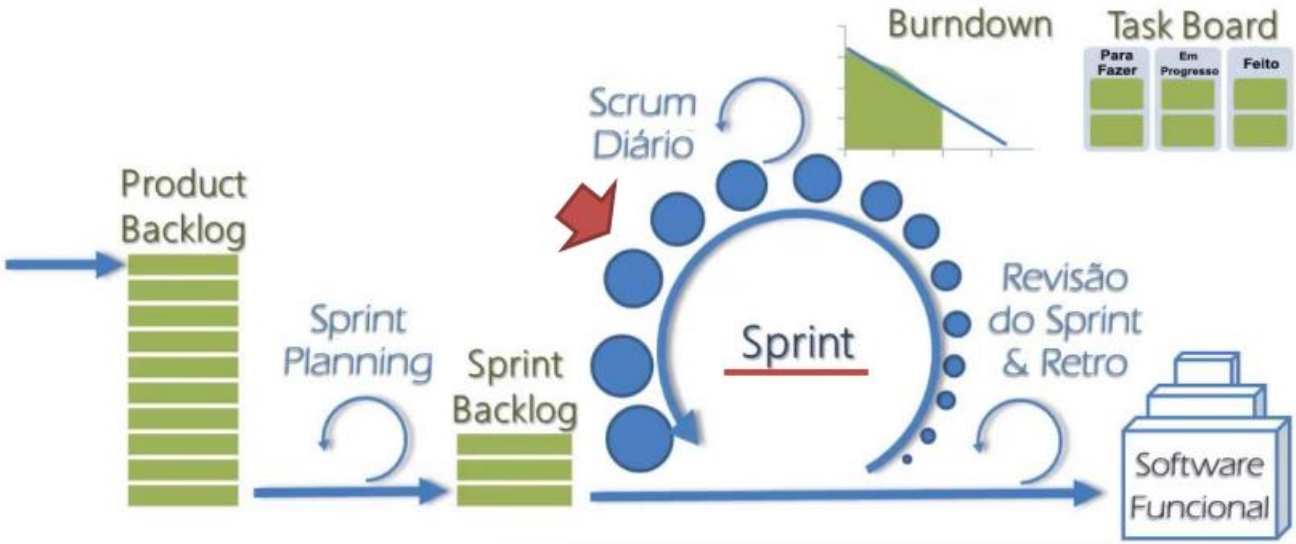
Sprint Planning Two
Duração de 4 horas.
A equipe define as tarefas necessárias e esforço para desenvolvimento. A **equipe** distribui as tarefas aos membros.
Product Owner pode apenas estar envolvido para esclarecimento de dúvidas.
Scrum Master conduz.

CICLO DE VIDA SCRUM



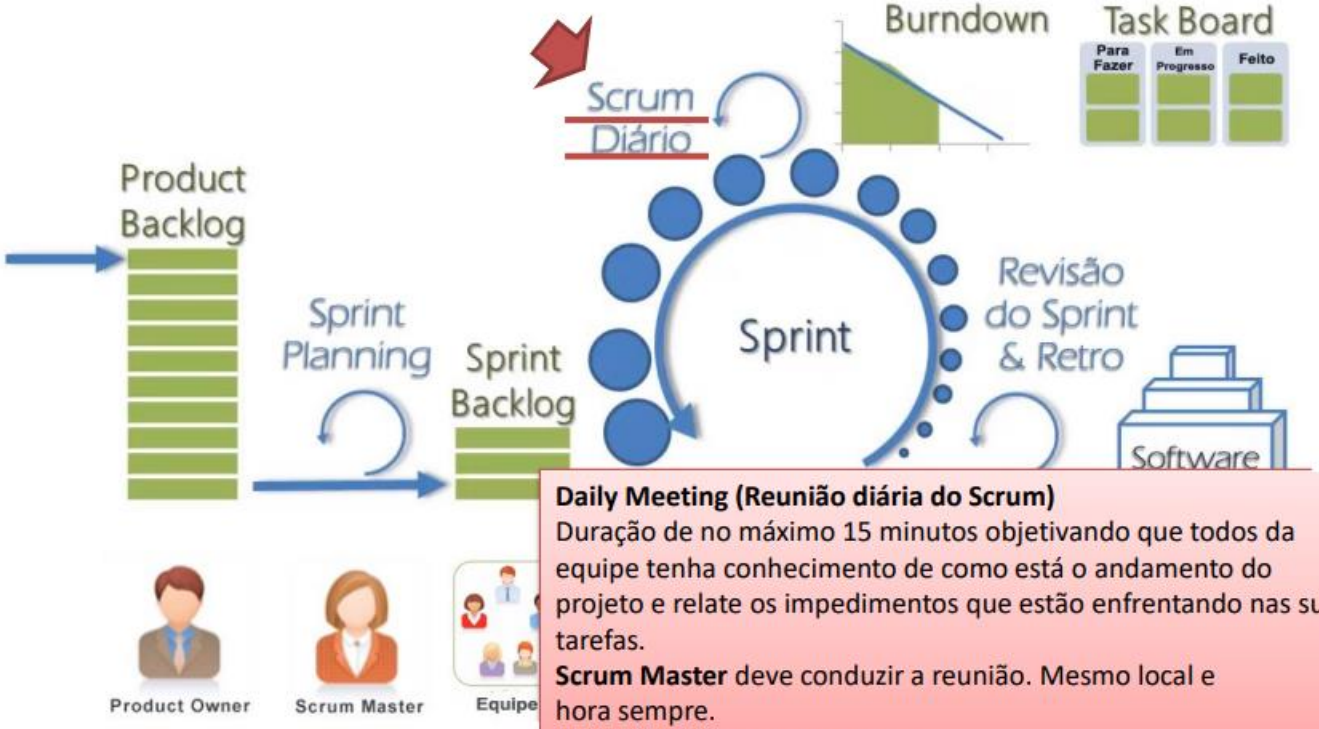
Sprint Backlog
Tarefas a serem desenvolvidas para garantir a conclusão da iteração

CICLO DE VIDA SCRUM



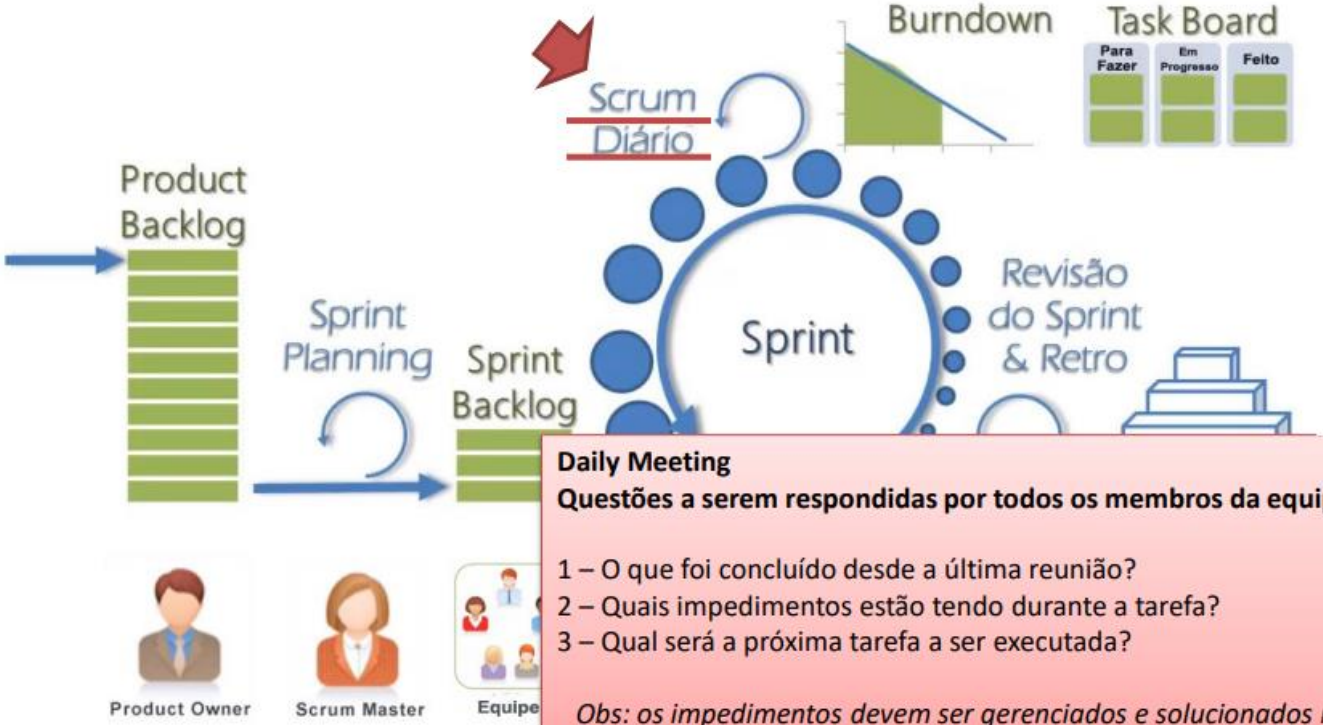
Sprint
Iteração de duração entre **uma semana** e no máximo **um mês** para desenvolver as tarefas do *Sprint Backlog* garantindo que o que foi selecionado será desenvolvido.
Um Sprint deve garantir um produto incremental e funcional para o cliente.

CICLO DE VIDA SCRUM



Daily Meeting (Reunião diária do Scrum)
Duração de no máximo 15 minutos objetivando que todos da equipe tenha conhecimento de como está o andamento do projeto e relate os impedimentos que estão enfrentando nas suas tarefas.
Scrum Master deve conduzir a reunião. Mesmo local e hora sempre.
Product Owner não participa.

CICLO DE VIDA SCRUM



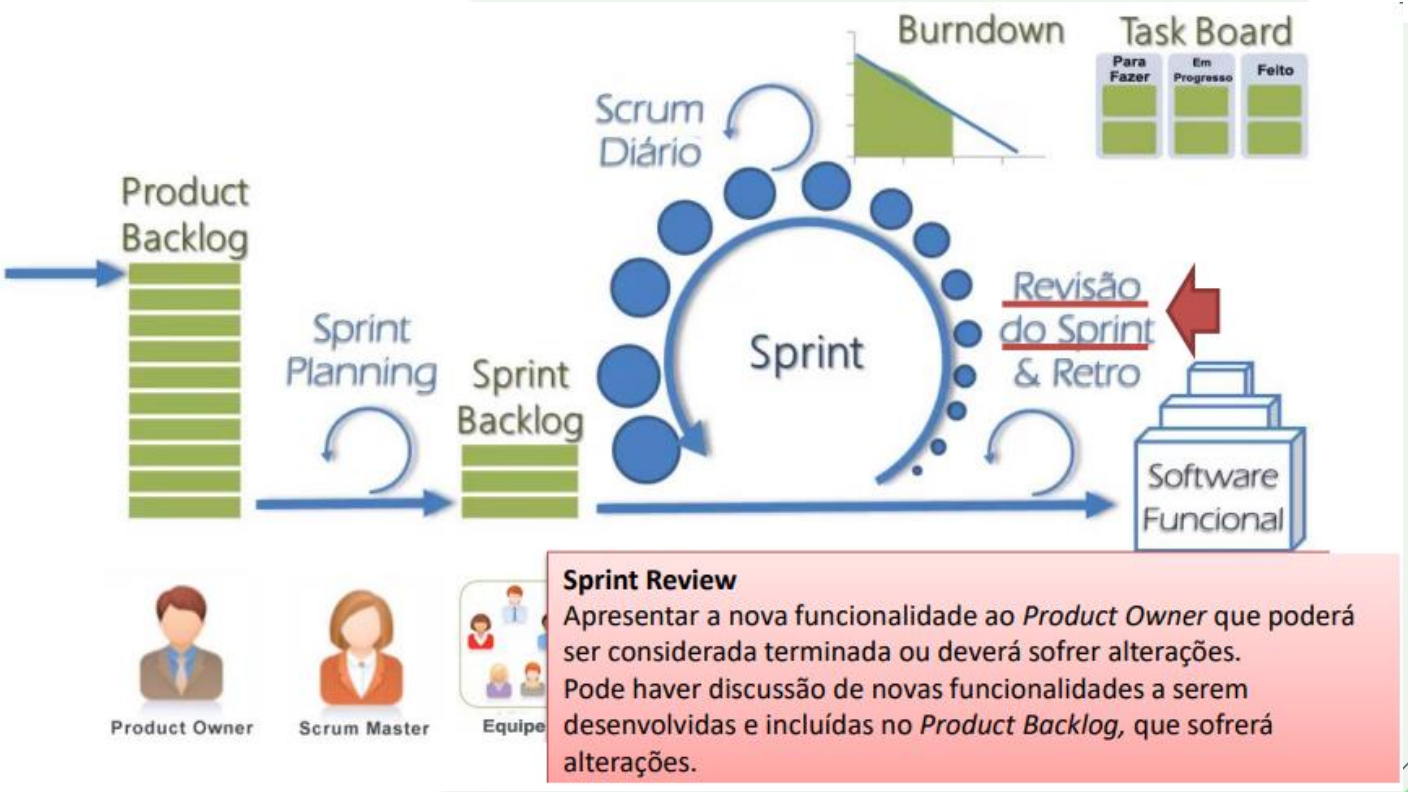
Daily Meeting

Questões a serem respondidas por todos os membros da equipe.

1 – O que foi concluído desde a última reunião?
2 – Quais impedimentos estão tendo durante a tarefa?
3 – Qual será a próxima tarefa a ser executada?

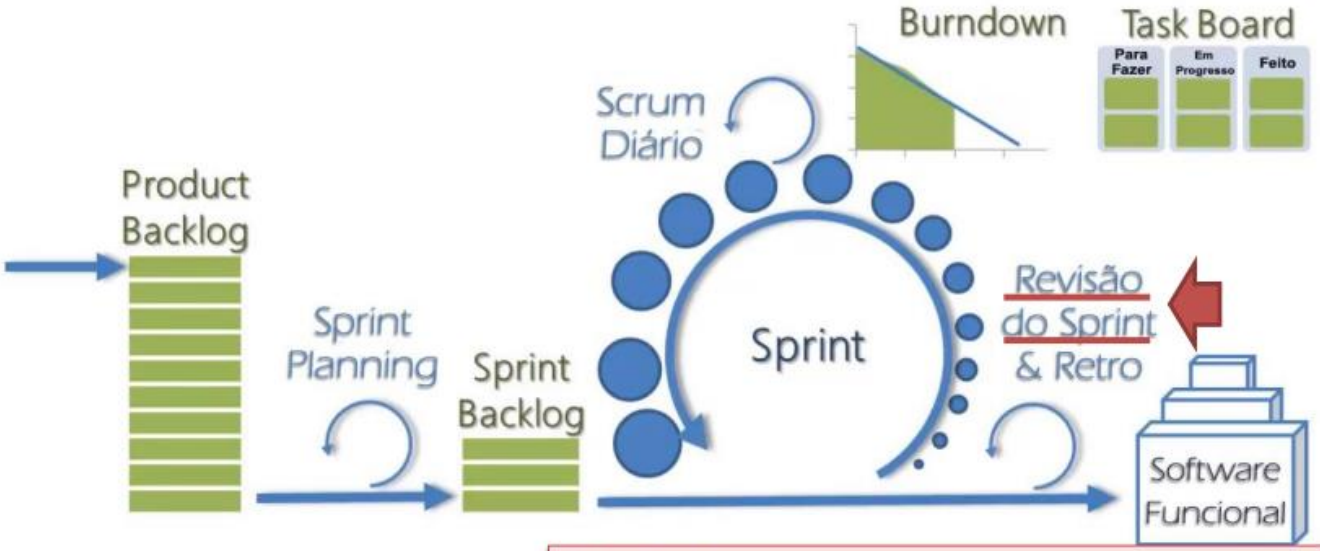
Obs: os impedimentos devem ser gerenciados e solucionados na reunião.

CICLO DE VIDA SCRUM



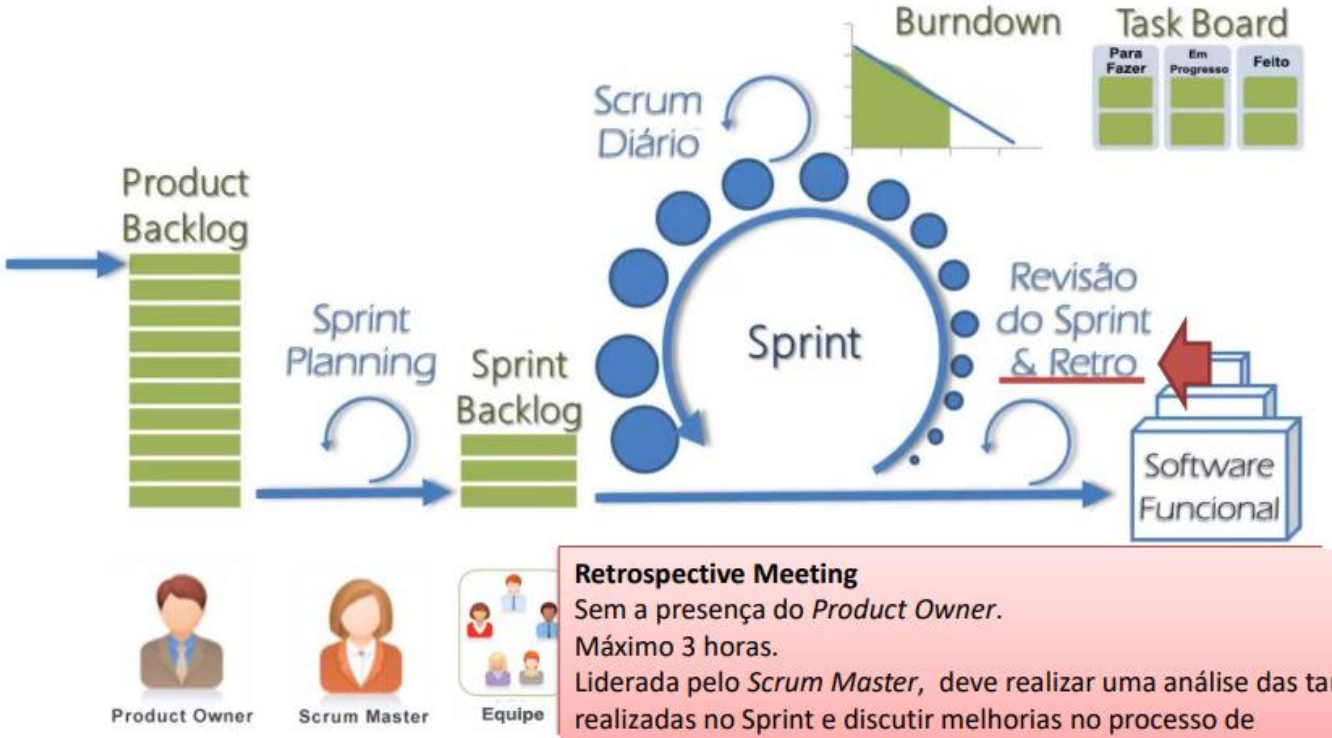
Sprint Review
Apresentar a nova funcionalidade ao *Product Owner* que poderá ser considerada terminada ou deverá sofrer alterações.
Pode haver discussão de novas funcionalidades a serem desenvolvidas e incluídas no *Product Backlog,* que sofrerá alterações.

CICLO DE VIDA SCRUM



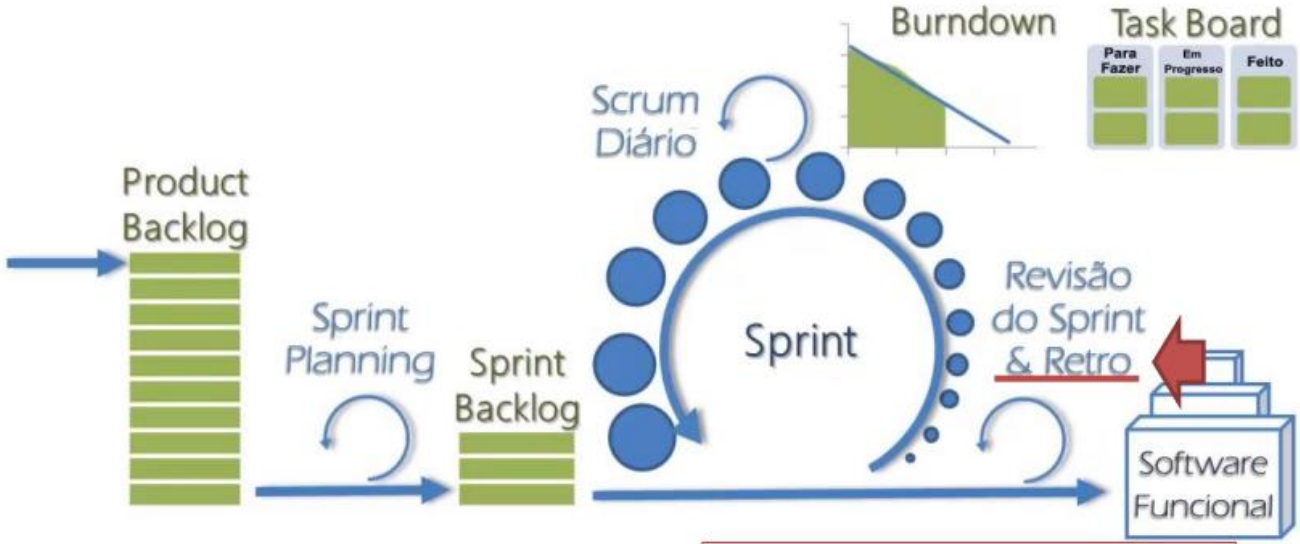
Sprint Review
Duração de no máximo 4 horas no último dia do Sprint.
A condução é de responsabilidade do **Scrum Master**.
Menos de uma hora de preparação.
Não usar slides (ppt). Deve demonstrar a funcionalidade em utilização simulada (**teste de aceitação**).

CICLO DE VIDA SCRUM



Retrospective Meeting
Sem a presença do *Product Owner*.
Máximo 3 horas.
Liderada pelo *Scrum Master*, deve realizar uma análise das tarefas realizadas no Sprint e discutir melhorias no processo de desenvolvimento.

CICLO DE VIDA SCRUM



Retrospective Meeting

Perguntas a serem respondidas:

1 – O que aconteceu de bom durante o último Sprint?

2 – O que pode ser melhorado para o próximo Sprint?

REFERÊNCIAS BIBLIOGRÁFICAS

- PAZOTI, M. A.; **Processos de Software.** Faculdade de Informática de Presidente Prudente (FIPP), 2021.

- PRESSMAN, R. S.; MAXIN, B. R. **Engenharia de software: uma abordagem profissional.** 8ª Edição. McGraw Hill Education, 2016.

- SOMMERVILLE, I. **Engenharia de software.** Pearson Education, 2011.

Unoeste | EAD